

DIA Project Report

Ayca Aksoy 520231

Baris Can Göktas 522149

SETUP

REQUIREMENTS: Jupyter Notebook, Python 3.11, Docker

1. Install Jupyter Lab.
2. Extract the project zip.
3. Make sure the /opt folder exists on your Mac : *sudo mkdir -p /opt*
4. Create a link named 'spark-data' that points to your real data folder. This makes /opt/spark-data on your Mac lead to
/Users/{username}/path/to/your/data:
sudo ln -s /Users/aycaaksoy/Dev/path/to/your/data /opt/spark-data
5. Verify it works: */opt/spark-data/DBahn-berlin*
6. Open a terminal in the same folder with docker-compose.yaml
7. On the terminal run the command: *docker compose up -d*
8. Open “./path/to/your/code” in Jupyter Lab.

TASK 1 Star Schema And ETL

1 Star Schema

Dimension tables

dim_stations (station metadata; one row per EVA station)

Primary key: *eva_number* (BIGINT)

Attributes: *station_name*, *latitude*, *longitude*, *category*

dim_trains (train identity / line metadata; one row per distinct train configuration)

Primary key: *train_key* (BIGSERIAL surrogate key)

Business attributes: *train_category*, *train_number*, *line_name*, *owner*

Uniqueness constraint: (*train_category*, *train_number*, *line_name*) is unique
(used for upserts / deduplication)

dim_time (hour-level time dimension; one row per hour)

Primary key: time_id = YYYYMMDDHH (BIGINT)

Attributes: full_date, year, month, day, hour, day_name, is_peak_hour
is_peak_hour is true for 07:00–09:00 and 17:00–19:00 (exclusive of 09 and 19).

Fact table

fact_train_stops (planned + actual stop information; one row per stop event)

Primary key: stop_id (VARCHAR)

Foreign-key columns (logical relationships):

station_eva_number → dim_stations.eva_number

train_key → dim_trains.train_key

time_id → dim_time.time_id

Measures / attributes:

Planned times: planned_arrival, planned_departure

Actual times (from changes, else fallback to planned): actual_arrival,
actual_departure

Delays (minutes): arrival_delay_min, departure_delay_min
platform

is_cancelled (BOOLEAN)

Indexes (performance):

Index on fact_train_stops(station_eva_number) for station-based aggregations

Index on fact_train_stops(arrival_delay_min) for delay-based filtering

The implementation keeps PKs and uniqueness constraints and uses dimension keys consistently. (Foreign keys are modeled through columns and join logic; they can optionally be enforced as FOREIGN KEY constraints if desired.)

2 ETL

Input data and file layout

The dataset consists of:

station_data.json for station metadata,
hourly timetable XML files under /timetables/{datehour00}/...,

15-minute change XML files under /timetable_changes/{datehourminute}/....

Parsing and transformation logic

Step A - Initialize DB schema

The pipeline starts by creating (dropping if existing) the star schema tables (dim_stations, dim_trains, dim_time, fact_train_stops) and creating performance indexes.

Step B - Load station dimension (station_data.json)

Parse the JSON.

For each station, select the EVA entry where isMain == true.

Insert (eva_number, station_name, latitude, longitude, category) into dim_stations using ON CONFLICT DO NOTHING.

Build a lookup dictionary in memory: station_lookup[station_name] = eva_number for later mapping of stop records to station keys.

Step C - Parse timetable changes XML (disruptions)

Read all change XML files recursively.

For each stop node <s id="...">:

 Extract "changed time" (ct) for arrival/departure (if present)

 Extract "changed platform" (cp) (if present)

 Compute is_cancelled:

 If arrival or departure has attribute clt (cancellation marker), mark the stop as cancelled.

Store the result in a dictionary keyed by stop_id:

 changes_map[stop_id] = {actual_arrival, actual_departure, changed_platform, cancelled}

This allows O(1) merge during timetable parsing.

Step D - Parse timetables XML (planned movements) and merge with changes

Read all timetable XML files recursively.

For each stop <s id="...">:

 Read planned arrival/departure pt

 Read train metadata from <tl>:

 c (category), n (number), o (owner), and the line name from ar/ dp attribute l

Merge actual values from changes_map (if available), otherwise fall back to planned:

```
actual_arrival = change.ct or planned_arrival
actual_departure = change.ct or planned_departure
platform = change.cp or planned_platform
is_cancelled = change.cancelled else False
```

Compute delays in minutes:

```
arrival_delay_min = (actual_arrival - planned_arrival) / 60
departure_delay_min = (actual_departure - planned_departure) / 60
(If missing times, delay defaults to 0.)
```

Step E - Load dimensions and fact table (incremental upserts)

For each merged stop record:

```
dim_trains upsert
Insert train meta with ON CONFLICT (train_category, train_number, line_name)
DO NOTHING RETURNING train_key
If not returned (already existed), query the existing train_key and reuse it.
dim_time upsert
Choose the time basis as the stop's planned arrival (if present) else planned
departure.
Create:
    time_id = int(datetime.strptime('%Y%m%d%H'))
    calendar attributes + day_name + is_peak_hour
Insert with ON CONFLICT DO NOTHING.
fact_train_stops insert
Determine station key via station_lookup[station_name]
Insert the stop record with ON CONFLICT (stop_id) DO NOTHING (so re-runs
are idempotent).
```

To improve performance batch commit is applied, the pipeline commits every 1000 inserted stops and then commits at the end.

TASK 2 Data Analytics

1 All SQL Queries are explained with comments in the code.

TASK 3 Spark ETL

1 Spark Session Configuration and Resource Allocation:

For the task 3 Large-Scale ETL and Analysis in Spark, Spark session configuration was hard to achieve. Our laptops have 16gb RAM, that is why we assign the resources considering this. Below some of the spark configuration is explained:

`.config("spark.driver.memory", "5g")` : Sets driver memory.

`.config("spark.driver.maxResultSize", "2g")` : Prevents OOM if you accidentally try to `.collect()` huge datasets. Safety limit set to 2GB.

`.config("spark.executor.memory", "4g")` : Sets memory available for storing RDDs, shuffles, and task execution. 4GB gives room for our XML parsing.

`.config("spark.executor.memoryOverhead", "1g")` : Sets off-heap memory for executor processes. It is used for JVM internals, Python workers, network buffers. 1GB prevents container kills.

`.config("spark.executor.instances", "1")` : It sets the number of executors which is a single one.

`.config("spark.executor.cores", "2")` : It sets CPU cores per executor. Each executor can run 2 tasks in parallel. This choice prevents memory pressure from too much concurrency.

`.config("spark.cores.max", "2")` : It sets maximum cores the application can use across all executors. It ensures no resource over-subscription on your laptop.

`.config("spark.sql.shuffle.partitions", "16")` : Sets default partition count for shuffle operations such as joins, groupBy, repartition etc. Our xml data is 4.41GB which means ~275MB per partition. This is a manageable configuration.

`.config("spark.sql.adaptive.enabled", "true")` : It enables Adaptive Query Execution. Adaptive Query Execution (AQE) is introduced in spark versions 3.x and it dynamically optimizes query plans during execution based on runtime statistics.

`.config("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128MB")` : Target size for each partition after coalescing since 128 MB is the most efficient size. It balances parallelism with overhead.

`.config("spark.sql.adaptive.skewJoin.enabled", "true")` : Handles data skew in joins automatically. If one partition is much larger than others, AQE splits it into sub-partitions to prevent stragglers.

`.config("spark.sql.files.maxPartitionBytes", "134217728")` : This is for optimizing the file reading. It sets the maximum size of each file partition when reading. Splits

large files into 128MB chunks for parallel reading. Prevents single tasks from processing huge XML files.

2 XML Reads

Spark-XML Library is used to handle nested xml structure, parsing and converting XML attributes to dataframe columns. Without this library, we would need to use lower-level RDD operations, which would be significantly more complex, error-prone and it is less supported in the newer versions of Spark.

The timetable schema is constructed based on the exact structure of timetable XML files. At the root level, we define `_station` as a `StringType` field with the underscore prefix indicating it's an XML attribute from the opening timetable tag. The schema then defines `s` as an `ArrayType`, which is important because each timetable file contains multiple stop elements, and Spark needs to understand this is a collection rather than a single item. Within each stop in the array, we've defined a nested `StructType` that contains three main components: arrival information (`ar`), departure information (`dp`), and train line information (`tl`). Each of these components is itself a `StructType` with its own fields. The arrival structure contains planned time (`_pt`), planned platform (`_pp`), and line (`_l`), while departure contains planned time and line information. The train line structure captures the category (`_c`), number (`_n`), and owner (`_o`) of the train. We marked the arrival and departure structures as nullable because not every stop has both arrival and departure information. Origin stations don't have arrivals, terminal stations don't have departures, and some stops might only record one or the other. Also, we choose to store time fields as `StringType` rather than `TimestampType` because the XML stores times in a compact format like "2401051430" (year-month-day-hour-minute in YYMMDDHHMM format) that needs custom parsing logic, which we handle later in the pipeline with the `to_timestamp` function using the "yyMMddHHmm" pattern. We use the `explode()` function applied to the `s` column in the read process. Before explosion, each row contains a single timetable with an array of potentially many stops. The explode operation flattens this structure by creating a separate row for each stop, which is necessary. Changes timetable schema follows similar principles but is structurally simpler because it only captures what has changed about scheduled trains rather than complete timetable information.

3 Data Transformation & Cleaning

In the transformation phase, first we joined planned timetable data with the station to see if there is any station with null `eva id`. Null `eva id` means that names did not match

so we checked one by one to see the issues. Since the raw station names in the timetable files often used abbreviations or slight variations compared to the station json file, we applied some normalisation prior to the join. `create_join_key` function is developed to create a column that lowercase all letters, handles prefix "Berlin-" and infix "Berlin" and remove all non-alphanumeric characters such as spaces, brackets, dashes. With using this function we make sure the values match properly before the join. Also unique cases were manually mapped; for example, "Yorckstr.(S1)" and "Yorckstr.(S2)" were resolved to their full unique station names, and "Berlin-Schöneweide" was corrected to "Berlin-Schöneweide Pbf". Following this, a series of regular expressions were used to standardize common German railway abbreviations: noise patterns like "(S)" were removed, "Betriebsbf" was expanded to "Betriebsbahnhof," "Hbf" to "Hauptbahnhof," "Bf" to "Bahnhof," and "Str." to "Straße".

4 Join

The join uses the generated `join_key` that we explained in the data transformation & cleaning part, as the common link between the two datasets. While joining the station and planned timetables data, broadcast join is used. In a standard Sort-Merge Join, Spark would need to shuffle both datasets, moving the massive timetable data across the network to align keys on different worker nodes. This is a very expensive and slow operation. If there is an extreme size difference between the two datasets, we can use broadcast join. This way we ensure that spark sends a full copy of the station table to every worker node once. Once the small station table is broadcasted, each worker can perform the join locally against its chunk of the timetable data without waiting for network transfers from other nodes and shuffling. This provides a significant performance boost. That is exactly our case. Also we `.cache()` the station dataframe since it has only 133 rows (very small size). Caching holds the data in cache which also helps in performance when reusing this dataframe.

Final join merges the planned and changes timetable data. This was achieved using a left join on the composite key of `stop_id` and `batch_id`. The `stop_id` provides a unique identifier for every specific train stop event, while the `batch_id` ensures that the updates strictly correspond to the correct data ingestion batch.

5 Writing to parquet

After creating the final dataframe, we write the data as a parquet file. Rather than defaulting to a static partition count (like the default 200 shuffle partitions), we dynamically calculate the coalesce number to optimize storage and read performance. The script calculates a `coalesce_factor` based on the actual volume of data, the number of unique `event_date` values. By setting the factor to approximately twice the number of

unique dates (with a minimum baseline of 8), the job ensures that each date partition receives a manageable number of reasonably sized files rather than hundreds of tiny ones. This approach helps us to avoid the "small file problem" which is very common in distributed systems, where excessive small files overwhelm the name node and degrade downstream query performance. The data is first repartitioned by event_date to align the physical distribution of data with the logical directory structure and then coalesced to reduce the number of output files per partition before being written out. This approach balances write parallelism with efficient read access for future reads.

6 Task 3.2 and 3.3 are explained with comments within the task 3 jupyter notebook.

TASK 4 Graph Based Analytics

Explained in the jupyter notebook.